Git & GitHub for Beginners

From your first commit to branches, Pull Requests, and a safety net for the AI era, all in one hands-on path

Press Space to begin · o for the slide overview

What you'll learn

One continuous path, in eight short stops, each building on the one before it.

0 **Get set up** · install Git + a GitHub account

01 **Fundamentals** · Git vs. GitHub

02 **The four locations** · where code lives

03 **The local workflow** · init → add → commit

04 **GitHub & syncing** · push, pull, fetch

05 **Branching & PRs** · merge & review

06 **Safety nets** · undo anything

07 **Setup & security** · SSH & secrets

08 **Best practices & tools** · habits & cheat-sheet

TIP

Every command is shown running on the slide that explains it. Press `o` any time to see all slides at once and jump around, handy as a reference later.

The whole map, at a glance

Eight stops in one picture, the path from the fundamentals out to the tools that build on them.

GLOSSARY

The words you'll meet

A quick reference for the key Git and GitHub terms in this deck. Skim it now for a feel, and come back whenever a word is fuzzy. Throughout, picture your project as a folder of files on your laptop, with the cloud as an online copy.

Glossary · the big picture

`Version control` A tool that saves your project's full history, so you can return to any past version.

`GitHub` A website that keeps a copy of your project in the cloud to back up and share it.

`Distributed` Every copy holds the complete history, so there is no single point of failure.

`Git` The free tool on your laptop that does the version control.

`Repository` A folder that Git tracks, including its full history. Also called a "repo".

`History` The ordered chain of all your commits, from your first save to your latest. Your current files are just the latest link; history is every link behind it.

Glossary · where your code lives

`Working Directory` The live folder on your laptop where you edit files.

`Local Repository` The hidden `.git` history on your laptop that stores every saved version.

`Commit` One saved version of your whole project, like a photo of it at that moment.

`Staging Area` A holding area where you pick which changes go into the next save.

`Remote Repository` The shared copy of your repo in the cloud on GitHub.

`Snapshot` Another word for what a commit saves, a full picture of your project.

Glossary · setting up and saving

`git config` One-time setup that signs your saves with your name and email.

`git clone` Downloads a full copy of an existing repo from GitHub.

`git status` Shows what has changed since your last save.

`git commit` Saves the staged changes as a new version.

`git init` Turns a normal folder into a Git repository.

`.gitignore` A file listing things Git should never track, like passwords.

`git add` Moves chosen changes into the staging area.

`git log` Lists your past commits, so you can scan your history.

Glossary · syncing with the cloud

`git push` Uploads your saved versions to GitHub.

`git pull` Downloads updates and merges them into your files. It is fetch plus merge.

`main` The primary, trustworthy version of your project. Older projects call it `master`.

`git fetch` Downloads updates into your history without changing your files.

`origin` The default nickname for your repo's copy on GitHub.

Glossary · branching and teamwork

`branch` A separate copy where you can try things without touching `main`.

`git merge` Combines one branch's work into another.

`Pull Request` A request on GitHub to merge your branch in, reviewed by others first.

`fork` Your own copy of someone else's repo, so you can improve it and suggest changes back.

`git checkout` Switches which branch or version you are working on.

`merge conflict` When two changes touch the same line and Git asks you which to keep.

`Merge Request` GitLab's name for a Pull Request.

`rebase` An advanced cleanup that rewrites history into one neat line. Safe to skip at first.

Glossary · undo and setup

`git restore` Throws away unsaved edits and returns a file to your last save.

`git reset` Rewinds your branch to an earlier commit. Local work only.

`git revert` Undoes a commit by adding a new one that cancels it, keeping history safe.

`git stash` Sets half-finished work aside, then brings it back later.

`SSH key` A one-time secure link so you push and pull without a password each time.

`Personal access token` A password-like code for connecting to GitHub over HTTPS.

Glossary · big files and storage

`monorepo` Keeping everything in one repository.

`binary file` A non-text file like a PDF, image, or dataset. Git saves each version in full but can't show what changed inside.

`Git LFS` Stores a big file you must keep as a small pointer plus a separate file. Free up to 1 GB.

`polyrepo` Splitting a project across many repositories.

`File / repo size limit` GitHub blocks any file over 100 MB and suggests repos stay under about 1 GB.

PART 1

Fundamentals

What Git is, what GitHub is, and why version control matters, before you type a single command.

Git is not GitHub

The single most common beginner mix-up. They work together, but they are not the same thing.

Git: the engine

A **version control system** that runs **locally** on your computer. It tracks every change, manages history, and works completely offline.

GitHub: the platform

A **cloud service** that **hosts** Git repositories online for backup, collaboration, and code review. It's one of several (GitLab, Bitbucket).

THE PHOTO ANALOGY

Git is your camera. Each **commit** takes a photo of your whole project, a saved version you can return to anytime. **GitHub is the photo-sharing site**, where you post the album so others can view it, comment, and build on it.

Version control = a time machine for your code

Before Git, "saving" meant `project_final_v2_ACTUALLY_final.zip`. Version control replaces that mess with something far more powerful.

Every save is recoverable

Git records a **snapshot** each time you commit. Rewind to any point in your project's history, and nothing is ever truly lost.

Distributed by design

Every developer holds a **full copy** of the entire history. No central server to fail, and it works offline.

NOTE

Git was created in 2005 by **Linus Torvalds**, the same mind behind Linux. Today it's used by roughly **94% of developers** (Stack Overflow, 2022): the de-facto industry standard.

Your undo button for the AI era

When an AI agent can rewrite twenty files in ten seconds, you aren't just writing code. You're managing **risk**. Git is the safety net.

Commit per task, not per session

Finished a feature, like a login modal or a refactor? Snapshot it **immediately**. Each commit is a working checkpoint you can return to.

One bad prompt away

AI is always one bad prompt from breaking everything. If the next prompt hallucinates, roll back to the last good commit in seconds.

TIP

The mindset shift: Git turns coding from a high-risk activity into one where you can **experiment boldly**, because every state is recoverable.

Git is local. GitHub is the cloud.

Keep that one distinction in your pocket.

Next: the four places your code lives on its way from your editor to the cloud.

PART 2

The four locations

To control your code, picture it moving through four rooms, each a different *state* of your project.

Four rooms your code lives in

1 · Working Directory

The **playground**, your project folder, where you actively edit.

State: **untracked** or **modified**

`git add` → Staging

2 · Staging Area

The **mirror**, where you group exactly the changes you want in the next snapshot.

State: **staged**

`git commit` → Local repo

3 · Local Repository

The **filing cabinet**, the hidden `.git` folder with your full history.

State: **committed**

`git push` → Remote

4 · Remote (GitHub)

The **cloud backup**, the shared mirror, safe from local hardware failure.

State: **synced**

`git pull` → Local repo

The flow: how a change travels

Every command's real job is to move your work between rooms, out to the cloud and back again.

`git pull` / `git fetch` bring the cloud down to local, while `git checkout` / `git restore` bring that history back into your working files.

The `.git` folder is the brain

When you run `git init` or `git clone`, Git creates one hidden folder: `.git`. It holds **everything**: every version, every commit message, all the history.

It's the whole repository

The files you see are just the *current* checkout. The full timeline lives compressed inside `.git`.

Decentralised

Because every clone carries the whole `.git`, there's no single point of failure. The cloud is just another copy.

WATCH OUT

Delete the `.git` folder and the project is no longer tracked. Git forgets it was ever a repository. Leave it alone, since you almost never touch it directly.

BEFORE YOU START

Get set up: Git + a GitHub account

Two one-time things the rest of this course assumes you already have. About five minutes now, and you're ready for Part 3.

Step 1: Install Git

Git is a free tool that runs **on your laptop**. Install it once per machine, then pick your OS:

macOS	`brew install git`	or `xcode-select --install`
Windows	`winget install Git.Git`	or installer at `git-scm.com`
Linux	`sudo apt install git`	Fedora: `sudo dnf install git`

Then confirm it worked:

```
git --version       # prints e.g. "git version 2.45.0" → you're ready
```

TIP

No version number? Close and reopen your terminal so it picks up the new command, then try again.

Step 2: Create a GitHub account

GitHub is the free website that stores your repos in the cloud. The account is **separate** from Git itself, and free for everyday use.

Sign up

1 · Go to `github.com` → **Sign up**

2 · Pick a **username** (`your-name`), the handle shown in every repo URL

3 · Choose a **password**

4 · Enter your **email** `you@personal.com`, which you'll reuse in Step 3

Verify, and you're in

Confirm the link GitHub emails you. The free plan covers unlimited public *and* private repos, which is everything this course needs.

NOTE

Git and GitHub are **different things**: **Git** runs on your laptop (Part 1), while **GitHub** is one place to keep a copy online. You'll link them securely with an SSH key in Part 7.

Step 3: set your ID card

Tell Git who you are. Use the **same email as your GitHub account** so your commits link back to your profile. This stamps every snapshot with your name.

```
git config --global user.name  "Your Name"         # display name (can differ from your username)
git config --global user.email "you@personal.com"  # the same email as Step 2
```

You do this **once per machine**. Every commit you ever make is then attributed to this identity.

> **NOTE**
>
> `--global` sets your identity for **every** project on your computer. Run the same command **without** `--global` inside a single repo to override it there, which is handy for a work email on work projects:
>
> ```
> git config --global user.email "you@personal.com"   # default for all repos
> cd ~/work/project                                    # go into one repo
> git config user.email "you@company.com"              # no --global → just this repo
> ```

HANDS-ON

See it work: push, pull, restore

Run these now and watch your files travel to GitHub and back, then survive a mistake. Don’t worry about the exact commands; Parts 3–4 explain each one.

Make file changes in Finder or your editor. In the terminal you only ever run `git` and `gh`.

Push: send your files up

In **Finder**, make a folder and create three text files inside it named `notes.txt`, `todo.txt`, and `readme.txt`. Open Terminal in that folder, then save your first snapshot:

```
git init                      # start tracking this folder
git add .                     # stage all three files
git commit -m "First commit"  # save the snapshot locally
git branch -M main            # name the default branch "main"
```

Now create the GitHub repo and push to it in one command:

```
gh repo create hello-git --public --source=. --push
```

> **TIP**
>
> First time only: `brew install gh`, then `gh auth login` (a quick browser sign-in). Now refresh **github.com** and you'll see all three files there. Your laptop just synced **up** to the cloud.

Commit again: stack a second snapshot

You rarely stop at one commit. In your editor, add a line to `readme.txt` and save, then take a second snapshot:

```
git add .                              # stage the change
git commit -m "Add a note to readme"   # snapshot #2, saved on your laptop
git log --oneline                      # two commits now, newest on top
```

COMMIT ≠ PUSH

That `commit` saved to your laptop only, we haven't pushed it. `git push` is a separate step that uploads commits to GitHub, and you choose when to run it.

Push now, or later?

`commit` and `push` are two different moves, so you decide when your work goes up to GitHub.

Push after the change

Collaborating, switching computers, or want an off-laptop **backup**? Run `git push` so GitHub has your latest commits. A good default habit.

No need to push yet

Just experimenting or mid-task on your own machine? Keep committing locally and **push later**, at a tidy stopping point. Nothing is lost, it's all saved on your laptop.

TIP

Rule of thumb: **commit often** (cheap, local, private); **push when it's worth backing up or sharing**. A whole stack of local commits can travel up in a single `git push`.

Time-travel: visit an old commit, then come back

Two commits means you can hop back to an earlier one, look around, then return, all without losing a thing.

```
git log --oneline       # copy the short ID of "First commit", e.g. a1b2c3d
git checkout a1b2c3d    # your files become snapshot #1 (no readme note yet)
git checkout main       # jump back to the latest commit, the note returns
```

TIP

You're only **visiting**: `git checkout a1b2c3d` shows the project exactly as it was at that commit, and `git checkout main` returns you to the newest one. Nothing is changed or deleted here, unlike `reset --hard` in Part 6.

Safety net: undo a mistake

Everything you commit is saved, so mistakes are reversible. Try it: in **Finder** delete `todo.txt`, and in your editor wipe the text in `notes.txt` and save. Then:

```
git status              # todo.txt deleted, notes.txt modified
git restore todo.txt    # bring the deleted file back
git restore notes.txt   # revert the wrecked file to its last snapshot
```

TIP

`git restore` rewinds your working files to the last commit, the snapshot that's also safe on GitHub. Nothing you've committed is ever truly lost.

Pull: bring a change down

Changes travel the other way too. Make an edit on GitHub itself:

1 · On **github.com**, open `notes.txt` → click the ✏️ pencil → add a line → **Commit changes**

2 · Back in your terminal, run the command below

3 · Open `notes.txt` on your laptop, and the new line is already there

```
git pull        # download GitHub's new commit to your laptop
```

> **NOTE**
>
> `git pull` = fetch + merge: it brings **down** commits that are on GitHub but not yet on your laptop. (It doesn't undo your local edits; that's what `git restore` is for.)

What just happened

Command	What it did
`git init`	started tracking your folder
`git add`	staged files for the next snapshot
`git commit`	saved a snapshot to local history
`git log --oneline`	listed your commits and their IDs
`gh repo create … --push`	made the GitHub repo and pushed
`git push`	sends new local commits up to GitHub
`git pull`	brings GitHub's commits down to your laptop
`git checkout <id>`	visits an old commit (`git checkout main` returns)

PART 3

The local workflow

From an empty folder to your first saved snapshot, using the commands you'll run every day.

Start tracking: `init` vs `clone`

Two ways a folder becomes a Git repository, and both create the hidden `.git`.

Brand-new project

Starting from your own folder? `git init` turns it into a fresh, empty repository, right where you are.

Existing project

Want a project that already exists online? `git clone <url>` downloads it, full history and all.

```
git init                    # start version control in the current folder
git clone <url>             # or copy an existing project from GitHub
```

TIP

Cloning with an **SSH** URL skips typing your password every time. More on SSH keys in Part 7.

`.gitignore` : your security guard

A plain text file listing what Git should **never** track. Create it before your first commit.

```
# .gitignore
.env                # secrets and API keys, never commit these
node_modules/       # huge, re-installable dependency folders
.DS_Store           # operating-system junk
```

WATCH OUT

Never commit secrets. Anything matched by `.gitignore` stays in your working directory and never travels to GitHub. Why this matters so much in the AI era: Part 7.

The daily loop

Three commands, run over and over. They are the heartbeat of working with Git.

```
git status                    # what changed? run this constantly
git add .                     # stage your changes for the next snapshot
git commit -m "Add login form"  # save the snapshot to your local history
```

THE GETTING-READY ANALOGY

`git add` is checking yourself in the **mirror**, where you can still swap a shirt (unstage a file) before you leave. `git commit` is walking out the door, and the snapshot is final.

Good commits, and reading history

A message in the present tense, describing one logical change, is a gift to future-you.

```
git commit -m "Add password reset email"   # present tense, one idea
git commit -am "Fix footer typo"            # -a: stage tracked files + commit in one step
git log --oneline                           # scan history compactly, find commit IDs
```

ONE COMMIT = ONE IDEA

Small, focused commits keep history readable and make rollbacks surgical.

CHECK BEFORE YOU COMMIT

Run `git status` first to make sure no `.env` or stray file is about to be saved.

What "history" really is

Your commits aren't a loose pile of saves. Each one is linked to the one before it, and that ordered chain **is** your history.

A chain of snapshots

Every commit points back to its parent, forming one timeline from your first save to now. `git log` walks that chain. Your **working files** are only the latest link; **history** is every link behind it.

History never forgets

A file you once committed stays in history **even after you delete it** from your folder. Removing it today doesn't erase it from older snapshots, which is exactly why a secret must never be committed in the first place.

DELETING A FILE IS NOT ERASING ITS HISTORY

Deleting a file only changes your **newest** snapshot; every older commit still holds it. Truly purging something means **rewriting history** (advanced), and once a commit is shared, rewriting it breaks everyone else's copy, so we `revert` instead (Part 6).

PART 4

GitHub & syncing

Connecting your local repo to the cloud, and keeping the two in step.

The cloud bridge

Your commits are safe locally, but only on one machine. **Pushing** copies them to GitHub: off-site backup, plus a place to collaborate.

```
git push origin main          # send your local commits up to the cloud
```

> **NOTE**
>
> Git is **distributed**: GitHub isn't the "boss", just another copy. Your full history still lives on your laptop even if the server goes down.

`fetch` vs `pull`: the classic confusion

Both bring news from GitHub. The difference is whether they touch the files you're looking at.

git fetch: look, don't touch

Downloads the latest from GitHub into your local *knowledge* of the remote. Your working files stay exactly as they are.

git pull: fetch + merge

Downloads **and** immediately merges those changes into your current files. `pull` = `fetch` + `merge`.

	Downloads changes	Changes your files
`git fetch`	✅	❌
`git pull`	✅	✅

`origin` and `main`: the vocabulary

Two words you'll see constantly when pushing and pulling.

origin

The default **nickname** for your remote on GitHub. `git push origin main` means "push to *origin*, the branch *main*."

main

The default **primary branch**, your stable, production-ready code.

> **NOTE**
>
> In 2020 the industry renamed the default branch from `master` to `main`. You'll still meet `master` in older projects, but they mean the same thing.

One golden rule: pull before you push

If a teammate pushed while you were working, your push will be rejected. Pull their changes in first, then send yours.

```
git pull origin main          # 1. get everyone's latest work
git push origin main          # 2. now send yours
```

TIP

This habit prevents most "rejected push" errors. It also catches merge conflicts early, while they're still small. More on conflicts next.

PART 5

Branching & PRs

Experiment safely on a branch, then merge after a human review.

Branching: a safe place to experiment

Working directly on `main` is risky. One bad change breaks the version everyone relies on. A **branch** is a separate copy where you can experiment freely.

A separate copy

`main` is the version everyone relies on, so protect it. A branch is a separate copy where you can try things, and if they break, `main` is untouched.

Fail without fear

If the experiment breaks, delete the branch and return to `main` as if nothing happened. If it works, merge it in.

TIP

In the AI era this is gold: let the AI try things on a branch. `main` stays clean no matter what it does.

The branch workflow

Three steps: branch off, do your work, merge back.

```
git checkout -b feature-login   # 1. create a new branch and switch to it
# ... edit, git add, git commit, exactly as before ...
git checkout main               # 2. switch back to the stable branch
git merge feature-login         # 3. combine your work into main
```

NOTE

`git branch` lists your branches and shows which one you're currently on.

Merge conflicts are decisions, not errors

A conflict happens when two branches changed the **same line** differently. Git can’t guess which you want, so it stops and asks.

```
<<<<<<< HEAD
const greeting = "Hello"
=======
const greeting = "Hi there"
>>>>>>> feature-login
```

A HUMAN DECIDES

This isn't a failure. It's Git handing you the decision. Keep the version you want (or combine them), delete the marker lines, then commit. The AI writes, and the human resolves.

Pull Requests: the quality gate

A **Pull Request (PR)** proposes merging your branch on GitHub. Before anything reaches `main`, a human reviews it.

See every change

GitHub shows each added (green) and removed (red) line. Teammates comment, suggest, and approve it line by line.

The AI safety gate

When an agent generates hundreds of lines instantly, the PR is where a person inspects them *before* they go live.

WHY 'PULL' NOT 'PUSH'

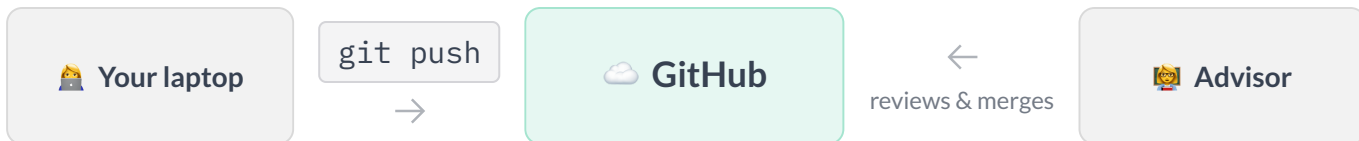
You're asking the project's owner to **pull** your branch in. GitLab calls the very same thing a "Merge Request".

IT'S A BUTTON

Opening a PR is one click on GitHub, not a command. What follows is the review conversation: comments, suggestions, approval.

Nobody touches your laptop

By the time you open a Pull Request, your branch is already pushed to GitHub. Your reviewer works entirely there, never on your machine.



GITHUB IS THE MIDDLEMAN

Everyone pushes their work up to GitHub and pulls updates down from it. You and your reviewer never connect directly, and nobody reaches into anyone else's computer.

Forking: contribute to repos you don’t own

You can’t push directly to someone else’s repository. **Forking** makes your own copy on GitHub, the launchpad for contributing back.

Fork

Copies someone else's repo into **your own GitHub account**, a repo you fully control.

Clone

Copies a repo to **your computer**. Clone your *fork* (not the original), so you can push to it.

TIP

The open-source flow: **fork** → **clone** your fork → branch → open a **Pull Request** back to the original.

Branch freely, merge deliberately

Isolate every experiment on a branch, then bring it home through a reviewed Pull Request.

One more term you'll hear: **rebase**, replaying your commits on top of the latest `main` for a cleaner, linear history. A polish step, not a beginner essential.

PART 6

Safety nets

Undo mistakes and recover work. The freedom to fail, on purpose.

The panic protocol

Something went wrong? There's a command for every kind of "undo".

Situation	Command
"I messed up my current, uncommitted edits."	`git restore .`
"I need to switch tasks but my work is half-done."	`git stash` ... `git stash pop`
"A commit broke things, rewind to a known-good point."	`git reset --hard <commit>`
"Undo a bad commit, but keep history honest."	`git revert <commit>`

WATCH OUT

`reset --hard` permanently discards changes. Use it on **local** work only, never on commits you've already pushed and shared.

Which undo? Ask how far it went

The right tool depends on one thing: how far the mistake has already traveled.

Still in your **unsaved edits**	→ `git restore`
In a **commit**, only on your laptop	→ `git reset --hard`
Already **pushed** to the team	→ `git revert`

WATCH OUT

Already pushed? Don't erase shared history, it breaks everyone else's copy. `git revert` adds a **new** commit that undoes the old one.

`revert` vs `reset`: two kinds of undo

Both undo a commit, but they tell very different stories.

git revert: the safe undo

Creates a **new** commit that reverses an old one. History stays intact and honest, so everyone sees the fix. Safe on shared branches.

git reset: the rewind

Moves your branch **back in time**, erasing later commits. Powerful but destructive, so keep it to local, unshared work.

TIP

Rule of thumb: **revert** in public (shared branches), **reset** in private (local only).

`git stash` : shelve work for later

Mid-task and suddenly need to switch branches? Don’t commit half-finished work, just stash it.

```
git stash           # tuck all uncommitted changes safely aside
git checkout main   # go fix that urgent bug on another branch
git stash pop       # come back and restore your work, right where you left off
```

> **NOTE**
>
> It sets your half-done work aside so you can deal with the interruption, then brings it back when you return.

PART 7

Setup & security

The one-time professional setup, and protecting your secrets, which matters more than ever.

Authentication: prefer SSH

GitHub needs to know it's really you before it accepts a push. There are two ways, and SSH keys are the smoother one.

HTTPS + token

Works everywhere, but you paste a personal access token (or get prompted), which is fiddly for everyday use.

SSH key

A one-time setup that identifies your machine to GitHub. After that, push and pull with no passwords at all.

```
ssh-keygen -t ed25519 -C "you@personal.com"   # generate a key, then add the .pub to GitHub
```

TIP

Set it up once per computer, and enjoy password-free `push` and `pull` forever after.

`.gitignore`, properly: protect your secrets

You met `.gitignore` in Part 3. Here's why it's non-negotiable in the AI era.

Keys get scraped in seconds

Your `.env` holds the API keys that power your LLMs. Push them to a public repo and bots find them almost instantly, racking up real charges.

History never forgets

Committing a secret then deleting it isn't enough. It stays in the Git **history**. The only safe move is to never commit it.

WATCH OUT

Add `.env` and key files to `.gitignore` **before** your first commit. If a secret ever does leak, rotate (regenerate) the key immediately.

Defaults worth setting once

A couple of one-time settings that make every future repo behave the way you expect.

Default branch = main

Make new repos start on `main` (not the legacy `master`), matching today's standard.

Your identity

The `user.name` / `user.email` you set during setup, attributed on every commit.

```
git config --global init.defaultBranch main   # new repos start on "main"
```

NOTE

These live in your global Git config. Set them on a fresh machine and forget about them.

PART 8

Best practices & tools

Good habits, the wider ecosystem, and a one-screen cheat-sheet to keep.

Best practices that pay off

Four habits that separate tidy repos from messy ones.

Commit small and often

Each commit, one logical change. Easy to read, easy to roll back.

Write clear messages

Present tense, says what changed and why. Future-you will thank you.

Protect main

Keep `main` stable and deployable. Do real work on branches and PRs.

Pull before you push

Sync the team's latest first to avoid needless conflicts.

Git for students: why bother

It pays off far beyond one class project.

Group projects

Each person works on their own branch and merges through a Pull Request. No overwriting.

Survive a bad week

Laptop dies before a deadline? If it's on GitHub, pull it onto any computer in minutes.

A hiring portfolio

Recruiters open your GitHub. A clean public repo proves you can do the work.

Free student tools

The Student Developer Pack: free Copilot, cloud credits, and more while enrolled.

Free hosting

GitHub Pages and Vercel host a page or live demo at a free, shareable link.

Not just for code

Notes, a thesis, LaTeX. Anything text-based gets the same history and backup.

Monorepo vs polyrepo

How many repositories should a project use? Two common answers.

Monorepo

Everything (frontend, backend, docs) in **one** repo. Simple to start and easy to see the whole project at once. Great for small teams.

Polyrepo

Each piece in its **own** repo. Independent deploys and fine-grained access control. Common at larger scale.

NOTE

There's no universally right answer. It's a trade-off between simplicity (mono) and independence (poly).

Tools & integrations

Git lives in a whole ecosystem. A quick tour of where you'll meet it.

Terminal

The command line. Full power, and what this deck taught.

GitHub Desktop

A friendly GUI: click to stage, commit, and push.

VS Code

Git built right into your editor's sidebar.

AI agents

Cursor, Claude, Copilot. Commit per task so you can always roll back.

Vercel · CI/CD

Push to `main` and your site goes live automatically.

Obsidian

Version your notes and "second brain", synced across devices.

Big files like PDFs and images

Git handles binary files too, just differently from your code.

It still saves every version

A PDF, an image, a dataset. Git keeps a full copy at every commit, so you can always get an old version back.

But they add up

Git can't show *what* changed inside a binary, and each saved version is a whole new copy, so a big file revised often makes the repo heavy.

GITHUB'S LIMITS

It blocks any single file over **100 MB**, and a repo should stay under about **1 GB**. A published Pages site is capped at **1 GB** too.

Big files: LFS, or just rebuild

Two ways to keep a big file from bloating your repo.

Git LFS

For a big file you **must** keep (a raw dataset). It stores a tiny pointer in Git and the real file in a separate place. Free up to 1 GB.

Or gitignore and rebuild

For a file you can **regenerate** (an export, a chart, a PDF). Ignore it and rebuild it, so it never bloats the repo at all.

TIP

This deck does the second one: its exported PDFs are gitignored and rebuilt by the deploy script, so there is nothing large to store and no LFS needed.

The command cheat-sheet

Command	From → To	Result
`git init`	folder → Working Dir	Start tracking and create the `.git` brain
`git status`	Working Dir	List modified vs. untracked files
`git add`	Working Dir → Staging	Stage changes for the next snapshot
`git commit`	Staging → Local Repo	Record a save point locally
`git push`	Local Repo → Remote	Send history up to GitHub
`git pull`	Remote → Working Dir	Fetch **and** merge cloud changes into files
`git fetch`	Remote → Local Repo	Update local knowledge, files untouched
`git checkout -b`	Local Repo → new branch	Create a branch and switch to it
`git stash`	Working Dir → shelf	Shelve unfinished work for later

The whole workflow on one screen

1 · Start

- `git init` or `git clone`
- Add a `.gitignore`
- Set your identity

2 · Daily loop

- `git status` to see what changed
- `git add .` to stage it
- `git commit -m "…"` to save it

3 · Branch

- `git checkout -b feat`
- work, then commit
- open a Pull Request

4 · Sync

- `git pull` before you push
- `git push origin main`
- goes live 🚀

Don’t just code, manage

You can now track history, sync with GitHub, branch fearlessly, and recover from any mistake.

Git turns coding from constant risk into complete control. It gives you the confidence to experiment, and the safety to build at scale.

Press `o` for the overview. This deck is yours to revisit as a reference.